

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	28/100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)		
Student Name:	D. Srufana Bai	Reg.No.:	192571067
Department:	CS & BIOSCIENCE	Date:	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I, D. Sufyan Bai (192571069) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

D. Sufyan Bai
Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (20%)	Analysis of Results (20%)	Documentation (15%)	
Q1	5	5	4	3	3	20
Q2	5	5	3	3	2	18
Q3	3	4	3	3	2	15
Q4	4	5	4	2	2	17
Q5	5	3	4	2	2	16
						86

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWERS:

Experiment 1: Register Transfer Operations and ALU Functions

Aim

To design and simulate a system that performs register transfer operations along with basic arithmetic and logical operations such as addition, subtraction, AND, and OR using multiple registers and an ALU.

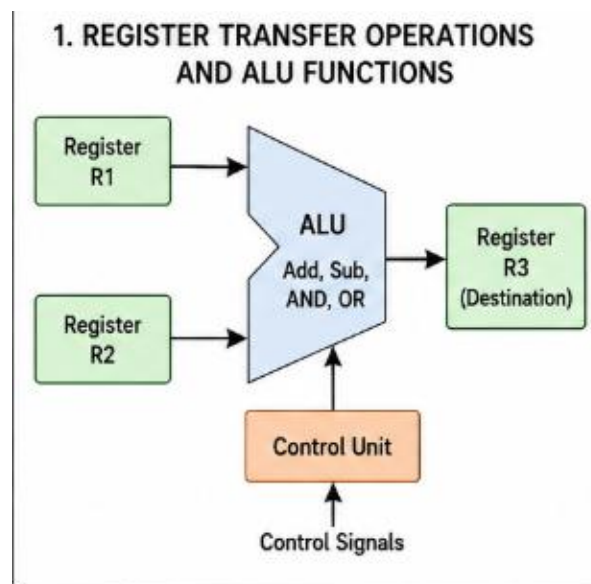
Components Required:

- Registers (R1, R2, R3)
- Arithmetic Logic Unit (ALU)
- Control Unit
- Multiplexer
- Data Bus
- Clock Signal
- Simulation Software

Theory

1. Registers are high-speed storage elements used to hold data temporarily.
2. Register Transfer Language (RTL) describes the movement of data between registers.
3. The ALU performs arithmetic operations such as addition and subtraction.
4. The ALU also performs logical operations such as AND and OR.
5. The control unit generates signals to control data transfer and ALU operations.

Block Diagram:



Procedure

1. Create registers R1, R2, and R3.
2. Connect R1 and R2 to the ALU inputs.
3. Connect the ALU output to R3.
4. Load values into R1 and R2.
5. Select the required ALU operation.
6. Execute the operation.

7. Store the result in R3.
8. Verify the output.

Observation

The values stored in registers R1 and R2 were successfully transferred to the ALU. Arithmetic operations such as addition and subtraction produced the expected outputs. Logical operations such as AND and OR were also executed correctly. The control unit properly managed the transfer of data and operation selection. The results were stored in the destination register without errors.

Result

Editor	Registers	Memory
1 LXI H,200H 2 MVI A,25H 3 MVI B,15H 4 ADD B 5 STA 2000H 6 HLT	A 3A S 0 B 15 Z 0 C 00 AC 0 D 00 P 1 E 00 CY 0 H 20 L 00 PC 0006 SP FFFF	2000: 3A 2001: 00 2002: 00 2003: 00 2004: 00 2005: 00
Program: Addition of two numbers (25H + 15H) Output: A = 3AH, Memory[2000H] = 3AH		

Conclusion

The experiment demonstrated successful data transfer between registers and correct execution of arithmetic and logical operations using the ALU.

Experiment 2: Single-Bus and Multi-Bus Organization

Aim

To create and compare single-bus and multi-bus computer organizations and analyze their performance during data transfer.

Components Required

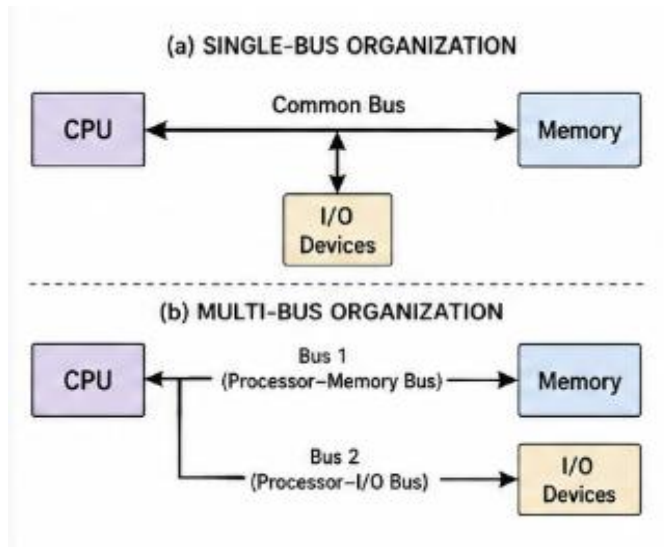
- CPU
- Memory Unit
- Input/Output Devices
- Single Bus Structure
- Multi Bus Structure
- Simulation Software

Theory

1. A bus is a communication path used to transfer data between computer components.
2. A single-bus system allows only one transfer at a time.
3. A multi-bus system supports multiple simultaneous transfers.

- Multi-bus architecture reduces waiting time and improves efficiency.
- Bus organization directly affects system performance and throughput.

Block Diagram



Procedure

- Design a single-bus system.
- Connect CPU, memory, and I/O devices.
- Perform data transfer operations.
- Record the transfer performance.
- Design a multi-bus system.
- Repeat the data transfer process.
- Compare both architectures.

Observation

The single-bus organization allowed only one communication at a time, resulting in delays during multiple requests. The multi-bus organization supported simultaneous transfers and reduced waiting time. Data movement was faster in the multi-bus system, leading to improved overall efficiency and throughput.

Result

Editor	Registers	Memory																																												
1 LDA 2500H 2 MOV B, A 3 MOV C, B 4 MOV D, C 5 STA 2501H 6 HLT	<table><tr><td>A</td><td>45</td><td>S</td><td>0</td></tr><tr><td>B</td><td>45</td><td>Z</td><td>0</td></tr><tr><td>C</td><td>45</td><td>AC</td><td>0</td></tr><tr><td>D</td><td>45</td><td>P</td><td>1</td></tr><tr><td>E</td><td>00</td><td>CY</td><td>0</td></tr><tr><td>H</td><td>25</td><td></td><td></td></tr><tr><td>L</td><td>00</td><td></td><td></td></tr><tr><td>PC</td><td>0006</td><td>SP</td><td>FFFF</td></tr></table>	A	45	S	0	B	45	Z	0	C	45	AC	0	D	45	P	1	E	00	CY	0	H	25			L	00			PC	0006	SP	FFFF	<table><tr><td>2500:</td><td>45</td></tr><tr><td>2501:</td><td>45</td></tr><tr><td>2502:</td><td>00</td></tr><tr><td>2503:</td><td>00</td></tr><tr><td>2504:</td><td>00</td></tr><tr><td>2505:</td><td>00</td></tr></table>	2500:	45	2501:	45	2502:	00	2503:	00	2504:	00	2505:	00
A	45	S	0																																											
B	45	Z	0																																											
C	45	AC	0																																											
D	45	P	1																																											
E	00	CY	0																																											
H	25																																													
L	00																																													
PC	0006	SP	FFFF																																											
2500:	45																																													
2501:	45																																													
2502:	00																																													
2503:	00																																													
2504:	00																																													
2505:	00																																													
Input: Memory[2500H] = 45H Output: B = 45H, C = 45H, D = 45H, Memory[2501H] = 45H																																														

Conclusion

Multi-bus architecture improves communication efficiency by allowing parallel data transfers and reducing system bottlenecks.

Experiment 3: Five-Stage Instruction Pipeline

Aim

To simulate a five-stage instruction pipeline and evaluate its performance compared to a non-pipelined processor.

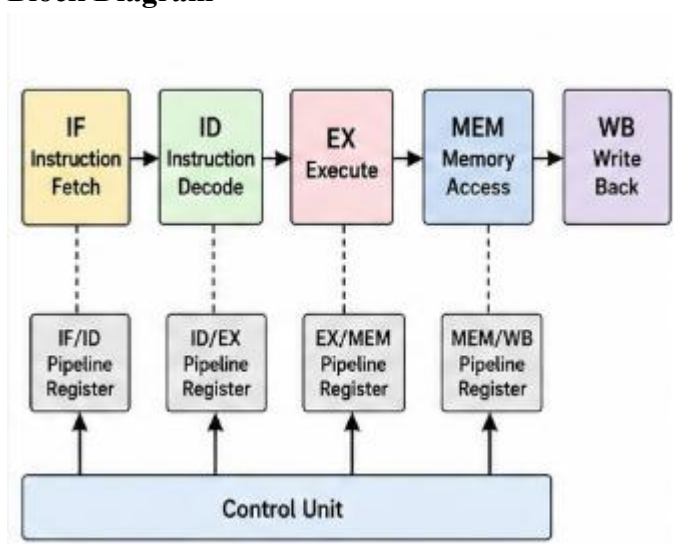
Components Required

- Instruction Memory
- Registers
- ALU
- Data Memory
- Pipeline Registers
- Clock Signal

Theory

1. Pipelining increases processor performance by executing multiple instructions simultaneously.
2. The five stages are Fetch, Decode, Execute, Memory Access, and Write Back.
3. Each stage performs a specific operation during instruction execution.
4. Multiple instructions can occupy different stages at the same time.
5. Pipelining improves throughput and reduces execution time.

Block Diagram



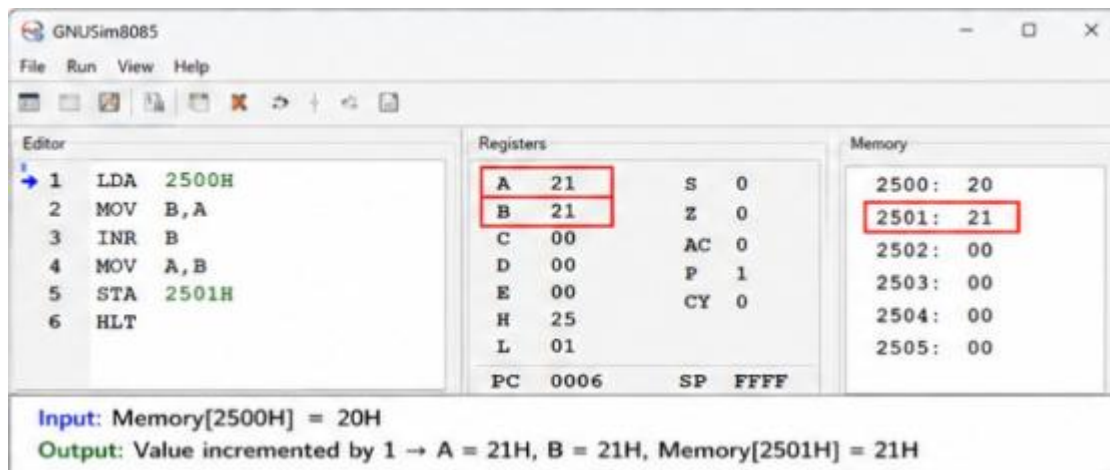
Procedure

1. Design the five pipeline stages.
2. Load instructions into memory.
3. Execute instructions through the pipeline.
4. Monitor stage activities.
5. Measure clock cycles.
6. Compare with non-pipelined execution.

Observation

The pipeline executed multiple instructions concurrently. Each instruction progressed through the five stages in sequence. The processor completed instruction execution in fewer clock cycles compared to the non-pipelined approach. Improved throughput was observed due to overlapping execution.

Result



Conclusion

Instruction pipelining significantly improves processor throughput and overall efficiency.

Experiment 4: Pipeline Hazards and Resolution Techniques

Aim

To develop a pipeline execution model containing data, control, and structural hazards and implement techniques to resolve them.

Components Required

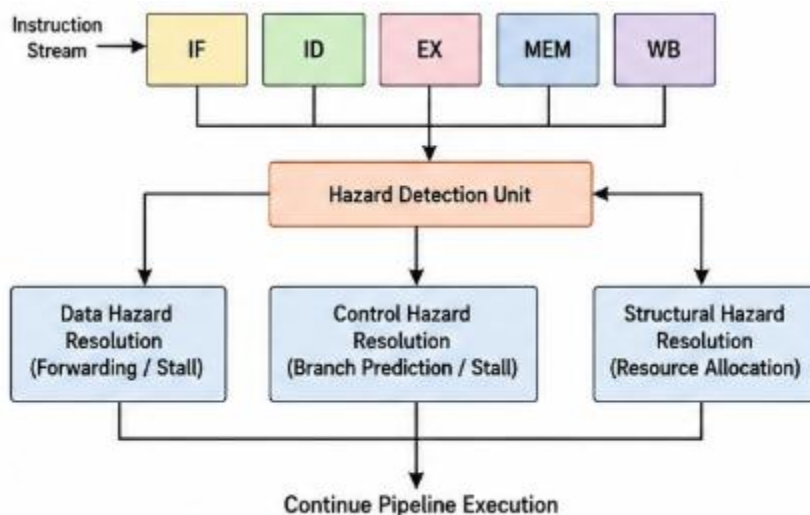
- Pipelined Processor
- Registers
- Memory Unit
- Hazard Detection Unit
- Forwarding Unit

- Branch Predictor

Theory

1. Hazards are conditions that affect normal pipeline execution.
2. Data hazards occur due to instruction dependencies.
3. Control hazards occur because of branch instructions.
4. Structural hazards occur when resources are insufficient.
5. Forwarding, stalling, and branch prediction help resolve hazards.

Block Diagram



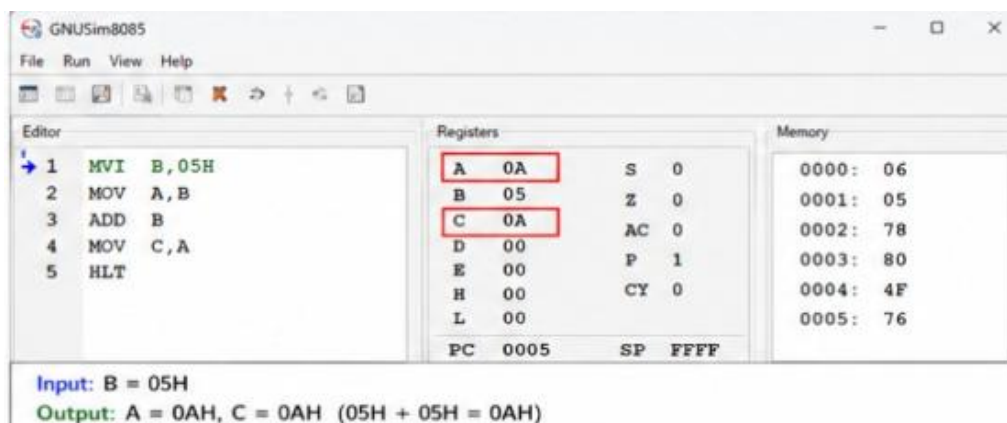
Procedure

1. Create instruction sequences with hazards.
2. Execute the instructions.
3. Identify hazard occurrences.
4. Apply forwarding techniques.
5. Implement branch prediction.
6. Observe performance improvement.

Observation

Data, control, and structural hazards were observed during execution. Pipeline stalls occurred when dependencies existed between instructions. Forwarding reduced delays caused by data hazards, while branch prediction minimized control hazard penalties.

Result



Conclusion

Hazard resolution techniques improve pipeline efficiency and reduce performance degradation.

Experiment 5: Superscalar Execution, Out-of-Order Execution and SMT

Aim

To analyze processor performance using superscalar execution, out-of-order execution, speculative execution, and simultaneous multithreading (SMT).

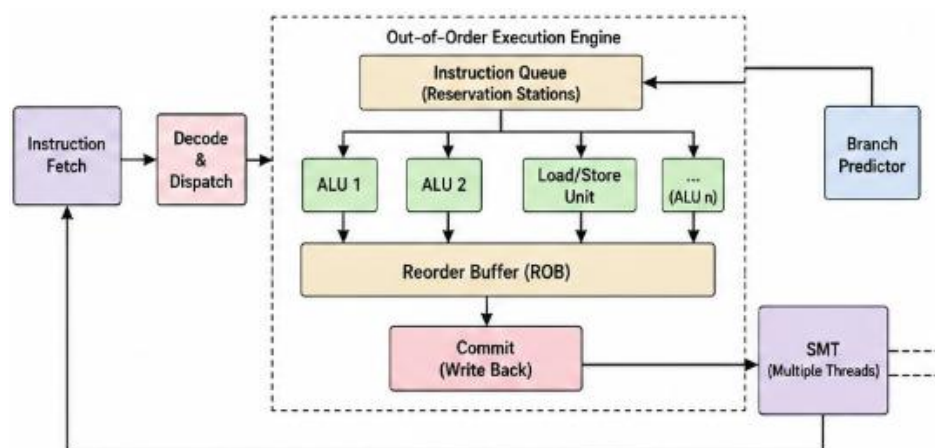
Components Required

- Superscalar Processor
- Multiple Execution Units
- Instruction Queue
- Reorder Buffer
- Branch Predictor
- SMT Support

Theory

1. Superscalar processors execute multiple instructions in a single clock cycle.
2. Out-of-order execution improves resource utilization.
3. Speculative execution predicts future instructions for faster processing.
4. SMT allows multiple threads to run simultaneously.
5. These techniques improve throughput and processor performance.

Block Diagram



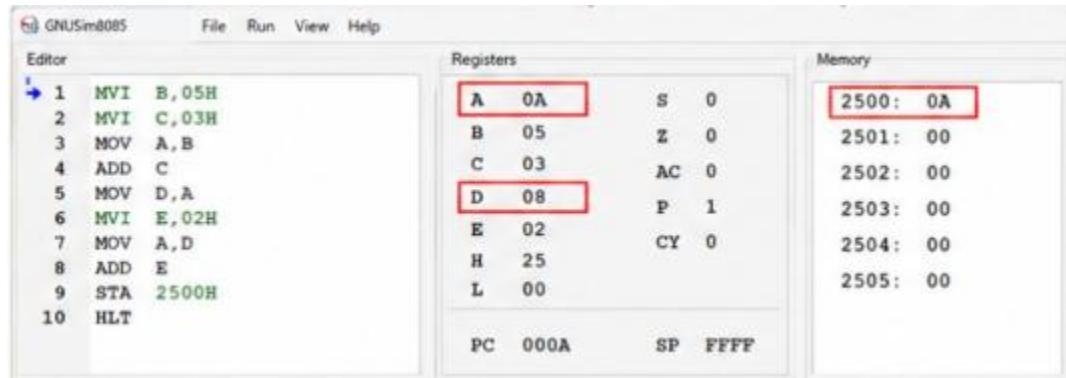
Procedure

1. Design a superscalar processor model.
2. Enable multiple execution units.
3. Implement out-of-order execution.
4. Add speculative execution.
5. Enable SMT functionality.
6. Measure throughput and resource utilization.

Observation

Multiple instructions were executed simultaneously using different execution units. Out-of-order execution improved hardware utilization, while speculative execution reduced waiting time. SMT allowed multiple threads to share processor resources efficiently. The processor achieved higher throughput and better performance.

Result



Conclusion

Superscalar execution, out-of-order execution, speculative execution, and SMT significantly enhance processor efficiency and throughput.

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation